

یادگیری کم‌نمونه برای تشخیص رفتار راننده با استفاده از شبکه‌های پروتوتایپی و مدل ترنسفورمری FEAT

شایان فتاحی

دانشجوی کارشناسی ارشد، گروه مهندسی کامپیوتر- گرایش هوش مصنوعی، دانشگاه آزاد اسلامی واحد جنوب، تهران، ایران

مهسا خانبانی

دانشجوی کارشناسی ارشد، گروه مهندسی کامپیوتر- گرایش هوش مصنوعی، دانشگاه آزاد اسلامی واحد جنوب، تهران، ایران

چکیده

تشخیص خودکار رفتار راننده یکی از اجزای کلیدی سامانه‌های ایمنی هوشمند محسوب می‌شود. با وجود پیشرفت‌های قابل توجه در یادگیری عمیق، بسیاری از روش‌های رایج تشخیص رفتار راننده متکی بر یادگیری نظارت‌شده و حجم زیادی از داده‌های برچسب‌خورده هستند. در سناریوهای واقعی، اگرچه ممکن است داده‌ی خام در دسترس باشد، اما استفاده عملی از آن معمولاً با محدودیت‌هایی نظیر هزینه‌ی برچسب‌گذاری، عدم توازن کلاس‌ها و نیاز به تعمیم‌پذیری به رفتارهای جدید مواجه است. از این رو، بررسی روش‌هایی که بتوانند در شرایط داده‌ی محدودشده به‌صورت هدفمند عملکرد مناسبی ارائه دهند، اهمیت ویژه‌ای دارد.

در این پژوهش، یک چارچوب یادگیری کم‌نمونه (Few-Shot Learning) برای دسته‌بندی رفتار راننده ارائه می‌شود که در آن از مجموعه‌داده‌ی Thesis_dataset2 استفاده شده است. اگرچه این مجموعه‌داده در حالت کامل دارای تعداد قابل‌توجهی نمونه در هر کلاس است، در این مطالعه با هدف شبیه‌سازی سناریوهای واقعی کم‌داده، تنها تعداد محدودی نمونه از هر کلاس به‌صورت کنترل‌شده انتخاب شده و فرایند آموزش و ارزیابی به‌صورت اپیزودیک (N-way, K-shot) انجام می‌شود. در این چارچوب، دو رویکرد مورد بررسی قرار می‌گیرند: (۱) شبکه‌ی پروتوتایپی مبتنی بر معماری Conv4 به‌عنوان یک روش کلاسیک متریک‌محور، (۲) مدل FEAT که با بهره‌گیری از ترنسفورمر، پروتوتایپ‌های کلاس را به‌صورت وظیفه‌محور و تطبیقی بازنمایی می‌کند.

نتایج تجربی نشان می‌دهد که مدل ProtoNet با وجود ساختار ساده، توانایی یادگیری مؤثر در سناریوهای داده‌ی محدودشده را دارد؛ با این حال، مدل FEAT به دلیل در نظر گرفتن تعامل میان کلاس‌ها و استفاده از سازوکار Self-Attention، عملکرد بهتری از نظر دقت و پایداری در شرایط مختلف K-shot ارائه می‌دهد. یافته‌های این پژوهش نشان می‌دهد که یادگیری کم‌نمونه نه‌تنها برای دیتاست‌های ذاتاً کوچک، بلکه به‌عنوان یک راهبرد مؤثر برای بهره‌برداری هوشمندانه از دیتاست‌های بزرگ در شرایط محدودیت عملی داده نیز کاربردی و قابل‌اتکا است. [6]

کلمات کلیدی: یادگیری کم‌نمونه، ترنسفورمرها، شبکه پروتوتایپی، توجه به خود، دسته‌بندی

۱- مقدمه

هر کلاس هستند، عملکرد ضعیفی نشان می‌دهند و دچار overfitting شدید می‌شوند. بنابراین توسعه روش‌هایی که بتوانند با «تعداد بسیار کم‌نمونه» به دقت قابل قبول برسند، به یک ضرورت تبدیل شده است.

در سال‌های اخیر، روش‌های سنتی (Classical Machine Learning) مانند HOG+SVM، LBP+kNN، SIFT+BoVW و مدل‌های مبتنی بر ویژگی‌های مهندسی‌شده، به‌عنوان راهکارهای اولیه مورد استفاده قرار می‌گرفتند. با وجود اینکه این روش‌ها در دیتاست‌های

در بسیاری از مسائل پردازش تصویر، یکی از چالش‌های اصلی، کمبود داده‌های برچسب‌دار در برخی کلاس‌ها یا سناریوهاست. در بسیاری از حوزه‌های عملی از جمله پایش صنعتی، شناسایی اشیاء خاص، بررسی محصولات، و سیستم‌های نظارتی فرایند جمع‌آوری داده‌های متنوع و پر حجم بسیار هزینه‌بر، زمان‌گیر یا حتی غیرممکن است. در چنین شرایطی، مدل‌های یادگیری عمیق معمولی که نیازمند هزاران نمونه برای

مدل کلاسیک Conv4 همراه با Prototypical Network، و مدل Transformer محور FEAT به عنوان یک متد مدرن و پیشرفته در Few-Shot Learning.

هدف ما بررسی این است که آیا استفاده از متا-لرنینگ و معماری‌های مبتنی بر attention می‌تواند generalization، دقت و توانایی یادگیری از داده‌های کم را به شکل چشمگیری نسبت به baseline های کلاسیک و شبکه‌های CNN افزایش دهد یا خیر. [3] نتایج آزمایش‌ها نشان می‌دهد که روش پیشنهادی مبتنی بر FEAT می‌تواند با اختلاف قابل توجهی عملکرد بهتری ارائه دهد، به ویژه در شرایط 1-shot و 2-shot که چالش اصلی بسیاری از مسائل واقعی است.

۲- پیشینه تحقیق

شناسایی و طبقه‌بندی رفتار رانندگان از جمله زیرمجموعه‌های حوزه حمل‌ونقل هوشمند است که می‌تواند در کاهش تصادفات نقش مهمی ایفا کند. در روش‌های مرسوم، اغلب سیستم‌های نظارت بر رفتار راننده بر مبنای پایش چهره و وضعیت بدن راننده با استفاده از دوربین استوار هستند. برای مثال مشاهده رفتارهای غیرعادی مانند خستگی یا مستی راننده معمولاً با مشاهده‌ی چشمان بسته، حالات چهره یا وضعیت بدنی او انجام می‌شود. این سیستم‌ها با مشکلاتی مانند نقض حریم خصوصی، نیاز به پردازش محاسباتی سنگین و وابستگی شدید به شرایط نوری مواجه‌اند. علاوه بر روش‌های مبتنی بر تصویر، برخی مطالعات از داده‌های حسی غیرمتحرک نیز برای طبقه‌بندی رفتار راننده استفاده کرده‌اند. به عنوان نمونه، داده‌های شتاب‌سنج سه‌محوره‌ی تلفن همراه برای ارزیابی رفتار راننده به کار رفته و شبکه‌های عصبی متعددی برای طبقه‌بندی این داده‌ها مورد استفاده قرار گرفته است.

در زمینه روش‌های یادگیری نظارت‌شده‌ی سنتی برای تشخیص رفتار راننده، الگوریتم‌هایی مانند ماشین بردار پشتیبان (SVM) و شبکه‌های عصبی کانولوشنی کلاسیک کاربرد فراوانی داشته‌اند. به عنوان مثال، در پروژه‌های مشابه، با استفاده از الگوریتم SVM موفق شدند راننده را به پنج سبک رانندگی طبقه‌بندی کنند (دقت حدود ۹۴/۶۷٪) و سپس با بهره‌گیری از قابلیت‌های شبکه‌های عصبی کانولوشنی (CNN) بر روی بازنمایی تصویری سیگنال‌های رانندگی، دقت تشخیص را به حدود ۹۹/۹۹٪ رساندند. در این مطالعه برای استفاده از شبکه‌های CNN، سیگنال‌های زمانی رانندگی به تصویر تبدیل شدند و سپس با یک CNN ساده معماری Conv4 طبقه‌بندی رفتار راننده انجام شد که نتایج بسیار دقیقی به دست آورد. [5]

کوچک قابل اجرا هستند، اما به شدت وابسته به انتخاب ویژگی‌ها بوده و در برابر تغییرات نور، زاویه، کیفیت و تنوع داده عملکرد پایداری ندارند. در نتیجه، این رویکردها برای مسائل پیچیده و واقعی که نیازمند درک معنایی عمیق هستند، کافی نیستند.

روش‌های مبتنی بر یادگیری عمیق، به ویژه شبکه‌های کانولوشنی (CNN)، تحول بزرگی در پردازش تصویر ایجاد کرده‌اند، اما وابستگی زیادی به داده‌های پر حجم دارند. حتی مدل‌های سبک وزنی مانند Conv4 نیز برای دستیابی به دقت بالا، به صدها یا هزاران نمونه نیاز دارند. وقتی تعداد نمونه‌ها به ۱ یا ۵ تصویر برای هر کلاس محدود شود، شبکه‌های معمولی قادر به یادگیری نمایش‌های مناسب نیستند و generalization به شدت افت می‌کند.

برای حل این مشکل، خانواده‌ای از روش‌ها با عنوان Few-Shot Learning معرفی شده‌اند که هدف آن‌ها توانایی مقایسه و انتقال دانش به کلاس‌هایی است که مدل در زمان آموزش هرگز ندیده است. یکی از رویکردهای تأثیرگذار در این حوزه، استفاده از meta-learning و یادگیری اپیزودیک است که از ساختار N-way K-shot تقلید می‌کند؛ یعنی همان ساختاری که مدل در زمان تست با آن مواجه می‌شود. روش‌هایی مانند Matching Networks، Prototypical Networks و Relation Networks نشان داده‌اند که با استفاده از نمایش‌های metric-based می‌توان با تعداد بسیار کم نمونه نیز به دقت‌های قابل قبول دست یافت.

از میان این روش‌ها، شبکه‌های Prototypical مبتنی بر معماری‌های CNN مانند (Conv4) به عنوان baseline قدرتمند و ساده در FSL شناخته می‌شوند. با این حال، این مدل‌ها در مواجهه با تنوع بالا میان کلاس‌ها و نیاز به انطباق پویا ضعف دارند. به همین دلیل، مدل‌های جدیدتر تلاش کرده‌اند نمایش‌های انعطاف‌پذیرتری بسازند.

یکی از پیشرفته‌ترین این روش‌ها، مدل FEAT است. با استفاده از Self-Attention مبتنی بر Transformer، نمایش‌های استخراج‌شده را در سطح episode تطبیق می‌دهد؛ به این معنا که embedding هر تصویر نه تنها بر اساس محتوای خودش، بلکه بر اساس رابطه با سایر نمونه‌های support و query اصلاح می‌شود. این ویژگی باعث می‌شود prototypes به صورت پویا (adaptive) شکل بگیرند و مدل بتواند تمایز میان کلاس‌های جدید را بسیار بهتر درک کند. بر همین اساس، FEAT یکی از قدرتمندترین مدل‌های metric-based FSL در سال‌های اخیر بوده و در مجموعه‌داده‌های چالشی مانند miniImageNet و tieredImageNet عملکرد چشمگیری داشته است.

در این پژوهش، ما دو رویکرد متفاوت را بر روی یک مسئله‌ی تصویری واقعی و با داده محدود مقایسه می‌کنیم :

یکی از محدودیت‌های کلیدی روش‌های فوق نیاز به حجم زیادی از داده‌ی برچسب‌دار برای آموزش است. یادگیری نظارت‌شده‌ی معمولی به‌ویژه هنگامی که از شبکه‌های عصبی پیچیده مانند CNN‌های عریض استفاده می‌شود، نیازمند صدها تا هزاران نمونه با برچسب برای هر کلاس است. اما در کاربردهای واقعی نظیر طبقه‌بندی رفتار راننده، جمع‌آوری داده‌های وسیع و دقیق دشوار، زمان‌بر و پرهزینه است. بدین ترتیب استفاده از روش‌های متداول بر روی داده‌های کم‌نمونه منجر به بیش‌برازش مدل می‌شود؛ به‌طوری که مدل در داده‌های آموزش خوب عمل می‌کند ولی در داده‌های جدید (خصوصاً شامل کلاس‌های جدید) شکست می‌خورد.

برای غلبه بر این محدودیت‌ها، حوزه یادگیری کم‌نمونه (Few-Shot Learning) مطرح شده است. یادگیری کم‌نمونه به معنای توانایی مدل برای انجام وظایف طبقه‌بندی جدید با اتکا به تعداد بسیار اندکی نمونه‌های برچسب‌خورده در هر کلاس (معمولاً بین یک تا ده نمونه) است. این رویکرد در واقع تقلیدی از توانایی انسان در یادگیری از چند مثال محدود است و می‌تواند هزینه و وقت لازم برای برچسب‌گذاری داده‌ها را کاهش دهد. در چارچوب متداول یادگیری کم‌نمونه، مسئله به صورت N-way K-shot تعریف می‌شود: در هر اپیزود آموزشی، N کلاس انتخاب شده و برای هر کلاس K نمونه ست پشتیبانی (support set)، و تعدادی نمونه‌ی آزمایشی کوئری (query set)، در نظر گرفته می‌شوند. مدل در هر اپیزود بر اساس آن ست‌های پشتیبانی آموزش می‌بیند و سپس سعی می‌کند نمونه‌های کوئری را طبقه‌بندی کند. بدین ترتیب مدل در دوره آموزش با شرایطی مشابه مرحله تست (حاوی کلاس‌های جدید و داده‌های محدود) مواجه می‌شود و می‌تواند بهتر تعمیم یابد.

روش‌های متریک‌محور یکی از رویکردهای موفق در یادگیری کم‌نمونه هستند. شبکه‌های متطابق (Matching Networks) و شبکه‌های پروتوتایپ از جمله معروف‌ترین این روش‌ها محسوب می‌شوند که بر یادگیری یک فضای جاسازی ویژگی متمرکزند تا بتوان با معیارهایی مثل فاصله یا شباهت، طبقه‌بندی را انجام داد. به طور خاص، شبکه‌های پروتوتایپ فرض می‌کنند در یک فضای جاسازی غیرخطی، نمونه‌های یک کلاس حول یک بردار نماینده (پروتوتایپ) قرار می‌گیرند. [1] در این روش با میانگین‌گیری از بردارهای جاسازی نمونه‌های ست پشتیبانی هر کلاس، بردار پروتوتایپ آن کلاس محاسبه می‌شود و نمونه‌های آزمایشی بر اساس نزدیک‌ترین پروتوتایپ به آن‌ها طبقه‌بندی می‌شوند. این روش سادگی محاسباتی دارد و برخلاف مدل‌های معمولی، دچار بیش‌برازش در شرایط کم‌نمونه نمی‌شود.

با پیشرفت تحقیقات few-shot، روش‌های مدرن‌تری مطرح شده‌اند که در عین حفظ ایده مبتنی بر فاصله و پروتوتایپ‌ها، قابلیت‌های قدرتمندتری ارائه می‌دهند.

به عنوان نمونه، مدل FEAT به تازگی معرفی شده است که در آن یک تابع مجموعه‌به‌مجموعه (set-to-set) مبتنی بر ساختار ترنسفورمر به کار گرفته می‌شود. در FEAT، برخلاف روش‌های کلاسیک که جاسازی‌ها را برای همه تسک‌ها ثابت در نظر می‌گیرند، ابتدا همه بردارهای جاسازی نمونه‌های ست پشتیبانی یک تسک را استخراج کرده و سپس با استفاده از یک ماژول ترنسفورمر، این جاسازی‌ها را به شکل task-specific ویرایش می‌کند. [2] در واقع هر بردار جاسازی که ورودی ترنسفورمر است، با توجه به سایر نمونه‌های پشتیبانی پردازش شده و یک بردار جاسازی اصلاح‌شده (contextualized embedding) تولید می‌شود. [2] این تغییرات ساختاری باعث می‌شود که پراکندگی نمونه‌های یک کلاس در فضا بیشتر به خوشه‌ی کلاس مربوطه نزدیک شود و تمایز میان کلاس‌های مختلف افزایش یابد (مطابق آنچه در FEAT نشان داده شده است). [2] افزون بر این، امتیاز اصلی این روش‌ها این است که توانایی تعمیم به کلاس‌های جدید را بدون نیاز به آموزش مجدد فراهم می‌کنند: در مرحله‌ی آزمون می‌توان تنها با چند نمونه برای هر کلاس جدید، پروتوتایپ‌های آن‌ها را ساخت و نمونه‌های تست را طبقه‌بندی نمود. به بیان دیگر، چارچوب متا-یادگیری کم‌نمونه این امکان را می‌دهد تا مدل بتواند با داده‌های اندک به خوبی به دسته‌های جدید تعمیم پیدا کند. [1]

۳- متودولوژی

در این پژوهش، هدف طراحی و ارزیابی یک سامانه‌ی یادگیری چندنمونه‌ای برای تشخیص الگو در شرایط کمبود داده بوده است. روش‌شناسی به‌گونه‌ای تدوین شده است که فرآیند یادگیری نه بر پایه‌ی اتکا به حجم زیاد داده، بلکه بر اساس توانایی مدل در سازگاری با مجموعه‌های داده‌ی کوچک و ناهمگون انجام شود. بدین منظور، دو معماری نماینده از خانواده‌ی روش‌های مبتنی بر فضاهای تعبیه (embedding space) انتخاب شده‌اند: معماری ProtoNet با تکیه بر استخراج ویژگی مبتنی بر Conv4، و معماری FEAT که یک رویکرد تطبیقی و مبتنی بر سازوکار توجه (Attention) است.

انتخاب این دو مدل به این دلیل صورت گرفته است که در ادبیات پژوهش، ProtoNet یکی از پایه‌ای‌ترین و پایدارترین روش‌ها برای ارزیابی شبکه‌های کم‌نمونه به‌شمار می‌رود و FEAT یکی از مدل‌هایی است که بهبودهای قابل‌توجهی نسبت به ProtoNet ارائه کرده است. مقایسه‌ی این دو معماری امکان تحلیل اثربخشی سازوکار تطبیق تعبیه‌ها را فراهم می‌سازد.

۳-۱-۲- سطح دوم

در سطح دوم، شبکه شروع به ترکیب ویژگی‌های استخراج شده در لایه‌ی قبل می‌کند تا الگوهای محلی پیچیده‌تری نظیر ترکیب لبه‌ها و ساختارهای تکرارشونده را مدل کند. در این مرحله، توزیع فعال‌سازی‌ها ممکن است در طول آموزش دچار تغییر شود که می‌تواند باعث ناپایداری گرادیان‌ها گردد.

برای رفع این مشکل، از نرمال‌سازی دسته‌ای (Batch Normalization) استفاده می‌شود. این تکنیک با نرمال‌سازی فعال‌سازی‌ها در هر دسته‌ی آموزشی، موجب تسریع همگرایی، کاهش حساسیت به مقداردهی اولیه و بهبود پایداری فرآیند آموزش می‌شود. همچنین پارامترهای قابل یادگیری γ و β به شبکه اجازه می‌دهند مقیاس و انتقال مناسب را برای هر ویژگی تنظیم کنند.

نرمال‌سازی دسته‌ای نیز با هدف کنترل توزیع ورودی لایه‌ها و کاهش نوسان گرادیان‌ها صورت می‌گیرد:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$y = \gamma \hat{x} + \beta$$

۳-۱-۳- سطح سوم

در لایه‌ی سوم، شبکه وارد فاز استخراج ویژگی‌های انتزاعی‌تر می‌شود. در این سطح، ترکیب سلسله‌مراتبی ویژگی‌ها منجر به شناسایی ساختارهای معنایی پیچیده‌تر مرتبط با کلاس‌ها می‌گردد. تابع فعال‌ساز ReLU با اعمال غیرخطی‌سازی، نقش مهمی در افزایش ظرفیت مدل ایفا می‌کند. این تابع با صفر کردن مقادیر منفی، از اشباع گرادیان جلوگیری کرده و موجب یادگیری نمایش‌های پراکنده (Sparse Representations) می‌شود که برای تعمیم بهتر در مسائل Few-Shot بسیار مؤثر است.

$$f(x) = \max(0, x)$$

۳-۱-۴- سطح چهارم

هدف اصلی تبدیل نگاشت‌های ویژگی با ابعاد مکانی به یک نمایش فشرده و ثابت بعد برای هر تصویر است. این کار از طریق عملیات Pooling (Max یا Average Pooling) Adaptive روش از روش Pooling استفاده می‌کنیم. Pooling باعث کاهش ابعاد مکانی، کاهش حساسیت به جابه‌جایی‌های کوچک و افزایش پایداری نمایش ویژگی‌ها

برای ایجاد یک بستر ارزیابی استاندارد، داده‌ها به صورت یکپارچه پیش‌پردازش و سپس در قالب وظیفه‌های اپیزودیک مورد استفاده قرار گرفته است. در این شیوه، هر اپیزود یک مسئله‌ی مستقل N-way K-shot محسوب می‌شود که شامل یک مجموعه‌ی پشتیبان و یک مجموعه‌ی پرس‌وجو است. چنین ساختاری فرض کلاسیک یادگیری نظارت شده را کنار می‌گذارد و یک چارچوب تطبیقی را برای مدل فراهم می‌کند که در آن، مدل باید در هر اپیزود تنها بر اساس تعداد معدودی نمونه، ساختار فضای کلاس‌ها را بیاموزد و سپس دسته‌بندی نمونه‌های جدید را انجام دهد.

۳-۱-۱- معماری ProtoNet

برای کل فرآیند، ابتدا نگاشت‌های ویژگی توسط مؤلفه‌ی استخراج‌گر پیاده‌سازی می‌شوند. در معماری ProtoNet، این مؤلفه توسط شبکه‌ای چهار لایه‌ای از نوع Conv4 پیاده‌سازی شده است. این شبکه با وجود سادگی ظاهری، از منظر نظریه‌ی پردازش سلسله‌مراتبی سیگنال‌ها قابلیت استخراج ویژگی‌های چندسطحی را دارد. هر لایه از این شبکه شامل یک عملیات کانولوشن با هسته‌های 3×3 ، به دنبال نرمال‌سازی دسته‌ای و تابع فعال‌ساز ReLU است. استفاده از هسته‌های کوچک 3×3 باعث می‌شود شبکه بتواند الگوهای محلی را با دقت بالا شناسایی کرده و در عین حال تعداد پارامترها کنترل شود؛ مسئله‌ای که در سناریوهای داده کم اهمیت ویژه‌ای دارد. تجمع این بلوک‌ها نه تنها موجب افزایش ظرفیت استخراج ویژگی می‌شود، بلکه از طریق عملیات پولینگ بیشینه‌گر موجب کاهش ابعاد و افزایش مقاومت شبکه در برابر جابه‌جایی می‌گردد. [4]

۳-۱-۱-۱- سطح اول

در لایه‌ی اول، هدف اصلی استخراج ویژگی‌های کم‌سطح نظیر لبه‌ها، گوشه‌ها و بافت‌های ساده است. این ویژگی‌ها بنیان ساختارهای پیچیده‌تر در لایه‌های بعدی را تشکیل می‌دهند. عملیات کانولوشن در این سطح با اعمال فیلترهای قابل یادگیری روی نگاشت ورودی انجام می‌شود. هر فیلتر با ترکیب خطی مقادیر همسایگی محلی و کانال‌های ورودی، یک نگاشت ویژگی جدید تولید می‌کند. این فرآیند امکان تشخیص الگوهای مکانی ساده را فراهم می‌سازد.

ریاضیات عملیات کانولوشن برای هر فیلتر به صورت زیر تعریف می‌شود:

$$Y_k(i, j) = \sum_{c=1}^C \sum_{m=1}^3 \sum_{n=1}^3 X(i+m, j+n, c) W_k(m, n, c)$$

می‌شود. در نتیجه، بردار ویژگی نهایی اطلاعات مهم تصویر را بدون وابستگی شدید به مکان دقیق اجزا حفظ می‌کند.

$$y(i, j) = \max_{\substack{0 \leq p < 2 \\ 0 \leq q < 2}} x(2i + p, 2j + q)$$

خروجی Conv4 یک بردار ویژگی $z \in \mathbb{R}^{64}$ برای هر تصویر است که نمایش فشرده‌ای از محتوای تصویر در فضای تعبیه‌شده را فراهم می‌کند. در Prototypical Network، برای هر کلاس c ، یک پروتوتایپ به صورت میانگین بردارهای ویژگی نمونه‌های پشتیبان آن کلاس تعریف می‌شود. این میانگین به عنوان نماینده‌ی مرکزی کلاس در فضای ویژگی عمل می‌کند و نسبت به نویز نمونه‌های منفرد مقاوم است.

$$\mu_c = \frac{1}{K} \sum_{i=1}^K f_{\theta}(x_i^c)$$

برای طبقه‌بندی نمونه‌های پرس‌وجو، فاصله‌ی اقلیدسی بین بردار ویژگی نمونه و پروتوتایپ هر کلاس محاسبه می‌شود. استفاده از فاصله‌ی اقلیدسی مربعی با فرض خوشه‌بندی کروی در فضای ویژگی سازگار بوده و پیاده‌سازی ساده و مؤثری دارد.

$$d(q, c) = \|f_{\theta}(q) - \mu_c\|_2^2$$

در نهایت، برای محاسبه‌ی احتمال تعلق نمونه‌ی پرس‌وجو به هر کلاس، از تابع Softmax بر روی فاصله‌های منفی استفاده می‌شود. این کار فاصله‌ها را به یک توزیع احتمال معتبر تبدیل می‌کند و امکان آموزش شبکه از طریق بهینه‌سازی تابع هزینه‌ی آنتروپی متقاطع را فراهم می‌سازد.

$$P(y = c | q) = \frac{\exp(-d(q, c))}{\sum_j \exp(-d(q, j))}$$

۳-۲-۲- معماری Feat

مدل FEAT با هدف رفع یکی از محدودیت‌های اصلی Prototypical Networks طراحی شده است. در ProtoNet، پروتوتایپ هر کلاس صرفاً به صورت میانگین ساده‌ی بردارهای پشتیبان تعریف می‌شود و تعامل بین کلاس‌ها در فرآیند ساخت پروتوتایپ‌ها نادیده گرفته می‌شود. Feat با معرفی یک ماژول تطبیق مبتنی بر Transformer، این محدودیت را برطرف کرده و امکان مدل‌سازی روابط ساختاری بین کلاس‌ها را فراهم می‌کند.

۳-۲-۱- پیش‌پردازش داده‌ها و نرمال‌سازی ورودی

پیش از استخراج ویژگی، تصاویر ورودی تحت یک زنجیره‌ی تبدیل (Transform Pipeline) قرار می‌گیرند. این پیش‌پردازش شامل تغییر اندازه تصاویر به ابعاد ثابت 84×84 ، افزایش تنوع داده از طریق وارونگی افقی تصادفی و نرمال‌سازی کانال‌های رنگی است. نرمال‌سازی با استفاده از میانگین و انحراف معیار مجموعه‌داده ImageNet انجام می‌شود که موجب هم‌ترازی توزیع داده‌ها با مدل‌های از پیش آموزش‌دیده و افزایش پایداری آموزش می‌گردد.

$$x' = \frac{x - \mu}{\sigma}$$

۳-۲-۲- استخراج ویژگی و فضای تعبیه

در FEAT، همانند ProtoNet، یک شبکه‌ی کانولوشنی (Conv4) به عنوان استخراج‌گر ویژگی به کار گرفته می‌شود. این شبکه نگاشتی از فضای تصویر به یک فضای برداری کم‌بعد اما معنادار ایجاد می‌کند. پس اینکه چگونه عصبانی‌شدنمان را بروز بدهیم، یک رفتار اجتماعی است. اتفاقی که در بدن یک فرد کره‌ای در زمان اندوهگین شدن رخ می‌دهد، همان رویدادی است که در بدن یک فرد آلمانی روی می‌دهد. اما تجربه حسی این دو فرد و بروز احساسی آن به دلیل تفاوت‌های فردی و اجتماعی با هم متفاوت است.

برای هر تصویر ورودی x ، خروجی شبکه یک بردار تعبیه‌شده به صورت زیر است:

$$z = f_{\theta}(x), z \in \mathbb{R}^E$$

که در این پژوهش $E = 64$ در نظر گرفته شده است.

این فضا به گونه‌ای یاد گرفته می‌شود که نمونه‌های متعلق به یک کلاس به یکدیگر نزدیک و از نمونه‌های کلاس‌های دیگر دور باشند.

۳-۲-۳- محاسبه‌ی پروتوتایپ‌های اولیه

پس از استخراج بردارهای ویژگی برای نمونه‌های پشتیبان، برای هر کلاس یک پروتوتایپ اولیه به صورت میانگین تعبیه‌های مربوط به آن کلاس محاسبه می‌شود. این مرحله کاملاً مشابه ProtoNet است و نقش نقطه‌ی شروع فرآیند تطبیق را ایفا می‌کند.

برای هر کلاس c ، پروتوتایپ اولیه به صورت زیر تعریف می‌شود:

$$\mu_c^{(0)} = \frac{1}{K} \sum_{i=1}^K f_{\theta}(x_i^c)$$

این پروتوتایپ‌ها هنوز مستقل از یکدیگر هستند و هیچ‌گونه تعامل بین کلاس‌ها در این مرحله لحاظ نمی‌شود.

خروجی نهایی Transformer ، مجموعه‌ای از پروتوتایپ‌های تطبیق‌یافته است:

$$\mu_c^* = g_\phi(\mu_c^{(0)})$$

این پروتوتایپ‌ها اکنون وابسته به زمینه‌ی اپیزود بوده و اطلاعات بین‌کلاسی را در خود کدگذاری می‌کنند، که این ویژگی موجب بهبود تفکیک‌پذیری در مسائل Few-Shot می‌شود.

۳-۲-۸- محاسبه‌ی فاصله و تابع زیان

برای هر نمونه‌ی پرس‌وجو q ، فاصله‌ی آن با پروتوتایپ‌های تطبیق‌یافته محاسبه می‌شود. در این پژوهش از فاصله‌ی اقلیدسی استفاده شده است:

$$d(q, c) = \| f_\theta(q) - \mu_c^* \|_2$$

سپس، با اعمال Softmax روی فاصله‌های منفی، احتمال تعلق نمونه به هر کلاس محاسبه می‌شود:

$$P(y = c | q) = \frac{\exp(-d(q, c))}{\sum_j \exp(-d(q, j))}$$

تابع زیان به صورت Negative Log-Likelihood تعریف می شود که معادل آنتروپی متقاطع در این چارچوب است:

$$\mathcal{L} = -\log P(y = y_q \mid q)$$

۴- ارزیابی و دقت مدل

در این بخش، عملکرد دو مدل ProtoNet مبتنی بر معماری Conv4 و FEAT در مراحل مختلف آموزش، اعتبارسنجی و آزمون مورد تحلیل دقیق قرار می‌گیرد. هدف از این تحلیل، بررسی رفتار همگرایی، پایداری یادگیری، و توان تعمیم مدل‌ها در مواجهه با اپیزودهای نادیده است. برای این منظور، از نمودارهای تغییرات تابع زیان و دقت در طول اپیزودها و همچنین ماتریس‌های اغتشاش مرحله آزمون استفاده شده است.

۴-۱- تحلیل رفتار تابع زیان (Training vs Validation Loss)

۴-۱-۱-۴ مدل Conv4 (ProtoNet)

در مراحل ابتدایی آموزش، منحنی زیان آموزش Conv4 کاهش سریعی را نشان می‌دهد. این رفتار ناشی از سادگی ساختار مدل و نبود ماژول‌های

۳-۲-۴- تطبیق پروتوتایپها با Transformer Encoder

نوآوری اصلی FEAT در این مرحله رخ می‌دهد. برخلاف ProtoNet، پروتوتایپ‌های اولیه مستقیماً برای طبقه‌بندی استفاده نمی‌شوند، بلکه ابتدا وارد یک Transformer Encoder می‌شوند تا به‌صورت زمینه‌مند (Context-aware) تطبیق داده شوند.

پروتوتایپ‌ها به عنوان یک دنباله (Sequence) در نظر گرفته می‌شوند:

$$\mathbf{P} = [\mu_1^{(0)}, \mu_2^{(0)}, \dots, \mu_N^{(0)}]$$

این دنباله به ماژول Transformer داده می‌شود. که شامل چند لایه‌ی Encoder با مکانیزم Self-Attention چندسری (Multi-Head Attention) است.

۳-۲-۵- مکانیزم Self-Attention و مدل سازی روابط بین کلاس ها

در هر لایه از Transformer، مکانیزم Self-Attention به هر پروتوتایپ اجازه می‌دهد اطلاعات سایر کلاس‌ها را در نظر بگیرد. برای هر پروتوتایپ، سه بردار Query، Key و Value محاسبه می‌شود:

$$Q = PW_Q, K = PW_K, V = PW_V$$

امتیاز توجه بین پروتوتایپ‌ها به صورت ضرب داخلی نرمال شده محاسبه می‌شود:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

این فرآیند باعث می‌شود هر پروتوتایپ نه تنها نماینده‌ی کلاس خود، بلکه متأثر از ساختار کلی، اییزود باشد.

۳-۲-۶- شبکه‌ی Feed-Forward و نرمال‌سازی لایه‌ای

پس از مرحله‌ی Attention، خروجی هر لایه‌ی Transformer وارد یک شبکه‌ی Feed-Forward دو لایه‌ای می‌شود که با تابع فعال‌ساز ReLU غیرخطی‌سازی می‌گردد. این شبکه ظرفیت مدل را افزایش داده و امکان یادگیری تبدیل‌های پیچیده‌تر را فراهم می‌کند. در انتهای هر لایه، از Layer Normalization استفاده می‌شود که باعث تثبیت توزیع فعال‌سازی‌ها و بهبود همگرایی در سناریوهای داده‌کم می‌شود.

$$\text{LN}(x) = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

تطبیقی پیچیده است که باعث می شود مدل بتواند به سرعت الگوهای غالب اپیزودها را یاد بگیرد.

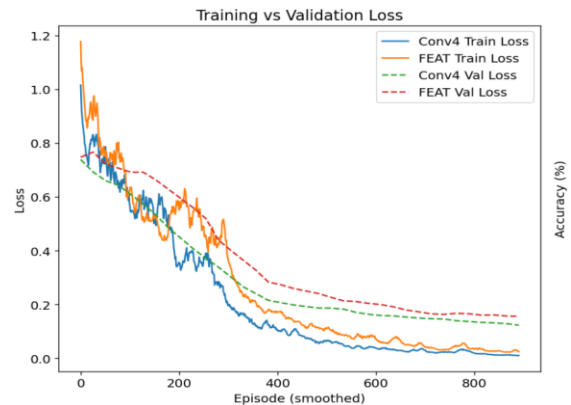
با افزایش تعداد اپیزودها، زبان آموزش به مقدار بسیار کوچکی نزدیک می شود، در حالی که زبان اعتبارسنجی با شیب ملایم تری کاهش یافته و در یک مقدار بالاتر تثبیت می شود. این شکاف پایدار بین زبان آموزش و اعتبارسنجی نشان دهنده تمایل مدل Conv4 به بیش برزش است.

این پدیده می تواند دلایلی مختلفی داشته باشد. می دانیم پروتوتایپها به صورت میانگین ایستا تعریف می شوند و هیچ مکانیزمی برای مدل سازی روابط بین کلاسها وجود ندارد پس صرفاً تصمیم گیری مبتنی بر فاصله های محلی می شود. در نتیجه، مدل در داده های آموزش عملکرد بسیار خوبی دارد، اما توانایی تعمیم آن به اپیزودهای جدید محدودتر است.

۲-۱-۴ رفتار مدل FEAT

در مقابل، مدل FEAT در ابتدای آموزش، مقدار زبان بالاتری نسبت به Conv4 دارد و کاهش زبان آن کندتر و نوسانی تر است. این رفتار کاملاً قابل انتظار بوده و ناشی از پیچیدگی بالاتر مدل و وجود ماژول Transformer است که نیازمند یادگیری روابط زمینه ای بین کلاسهاست. در واقع در این مدل با این رفتار مواجه می شویم که زبان اعتبارسنجی FEAT با روندی پایدار کاهش می یابد و فاصله ای بین زبان آموزش و اعتبارسنجی کمتر از Conv4 باقی می ماند.

این موضوع نشان می دهد که FEAT اگرچه دیرتر همگرا می شود، اما تعمیم پذیری بهتری نسبت به ProtoNet دارد. ماژول تطبیق پروتوتایپها باعث می شود تصمیم گیری مدل کمتر وابسته به نمونه های خاص آموزش و بیشتر وابسته به ساختار کلی هر اپیزود به صورت جداگانه باشد.



عکس ۴-۱ - نمودار تحلیل رفتار مدل (Loss) در حین آموزش برای دو مدل Conv4 و Feat و مقایسه آنها با یکدیگر

۲-۴ تحلیل رفتار دقت (Training vs Validation Accuracy)

۲-۴-۱ دقت مدل Conv4

منحنی دقت آموزش Conv4 به سرعت افزایش یافته و در مراحل میانی به نزدیکی ۱۰۰٪ می رسد. این رفتار نشان دهنده توان بالای مدل در حفظ نمونه های آموزش است که البته باید در نظر گرفت این دقت یادگیری در هر اپیزود است. با این حال، دقت اعتبارسنجی با اختلاف قابل توجهی پایین تر از دقت آموزش باقی می ماند. این شکاف ممکن است دلایلی داشته باشد. مدل Conv4 تصمیم گیری خود را بر اساس الگوهای خاص اپیزودهای آموزش تنظیم کرده است و یا تغییرات جزئی در اپیزودهای اعتبارسنجی منجر به افت عملکرد شود.

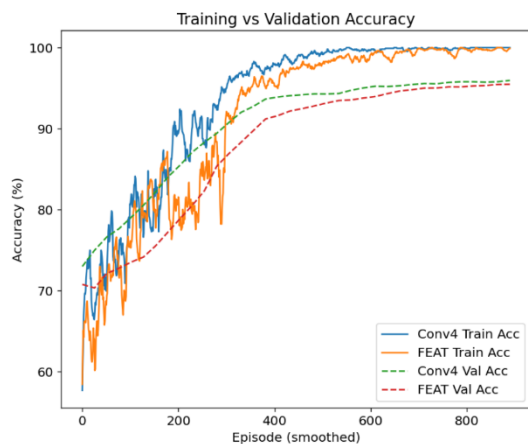
به بیان دیگر، مدل به جای یادگیری نمایش های تعمیم پذیر، به تطبیق محلی بیش از حد گرایش پیدا کرده است.

۲-۴-۲ دقت مدل FEAT

در مدل FEAT، افزایش دقت آموزش کندتر اما یکنواخت تر است. نوسانات اولیه دقت آموزش ناشی از پویایی ماژول Self-Attention در تطبیق پروتوتایپهاست که در ابتدای آموزش هنوز پایدار نشده است.

نکته کلیدی آن است که فاصله ای دقت آموزش و اعتبارسنجی در FEAT کمتر است و در ادامه دقت اعتبارسنجی به مقدار بالاتری نسبت به Conv4 می رسد.

این رفتار نشان می دهد که FEAT قادر است روابط بین کلاسها را در سطح اپیزود درک کرده و تصمیم های مقاوم تری نسبت به تغییر نمونه ها اتخاذ کند.



عکس ۴-۲ - نمودار تحلیل دقت مدل (Accuracy) در حین آموزش برای دو مدل Conv4 و Feat و مقایسه آنها با یکدیگر

۳-۴- جمع‌بندی تحلیلی برای بخش آموزش مدل

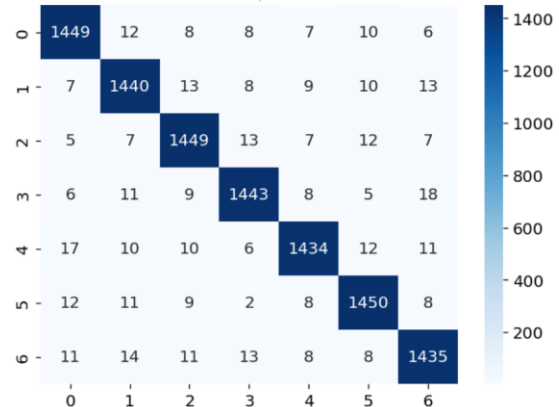
تحلیل همزمان دقت آموزشی و اعتبارسنجی نشان می‌دهد که هر دو مدل به دقت‌های بالا نزدیک به اشباع دست پیدا می‌کنند، با این تفاوت که مدل FEAT در برخی بازه‌ها دقت آموزشی بالاتری را تجربه می‌کند. این رفتار نشان‌دهنده‌ی ظرفیت بالاتر FEAT در مدل‌سازی روابط پیچیده بین کلاس‌ها در فضای ویژگی است. با این حال، فاصله‌ی مشاهده‌شده میان دقت آموزش و اعتبارسنجی در FEAT اندکی بیشتر از ProtoNet است که می‌تواند بیانگر حساسیت بیشتر این مدل نسبت به ساختار اپیزودهای آموزشی باشد.

۴-۴- تحلیل Confusion Matrix

طبق رفتار مدل در تصویر زیر، مدل نشان می‌دهد که مدل Conv4 تصمیم‌گیری خود را عمدتاً بر اساس شباهت‌های محلی در فضای ویژگی انجام می‌دهد و در مواردی که نمونه‌ها دارای ویژگی‌های مرزی یا همپوشان هستند، دچار خطای طبقه‌بندی می‌شود. در نتیجه، Conv4 اگرچه در سطح کلی عملکرد بسیار خوبی دارد، اما در مواجهه با نمونه‌هایی که در مرز بین کلاس‌ها قرار دارند، حساس‌تر عمل می‌کند.

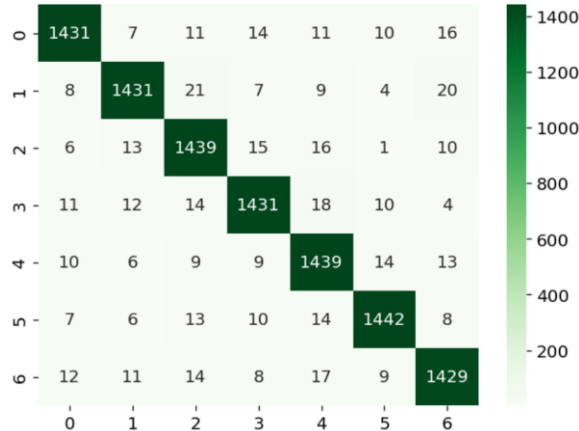
در صورتی که مدل FEAT تصمیم‌گیری خود را نه صرفاً بر اساس شباهت محلی، بلکه با در نظر گرفتن روابط زمینه‌ای بین تمام کلاس‌های اپیزود انجام می‌دهد. در برخی موارد، این موضوع منجر به بازتوزیع خطاهای بین کلاس‌ها می‌شود، بدون آن که دقت کلی کاهش یابد. این تفاوت نشان می‌دهد که FEAT حتی زمانی که دقت کلی مشابه Conv4 دارد، رفتار تصمیم‌گیری متفاوت و آگاهانه‌تری از ساختار مسئله Few-Shot ارائه می‌دهد.

Conv4 (Proto-like) Confusion Matrix
Precision=0.96, Recall=0.96



عکس ۴-۴-۱ - ماتریس درهم‌ریختگی مدل Conv4 برای بخش آموزش

FEAT Confusion Matrix
Precision=0.96, Recall=0.96



عکس ۴-۴-۲ - ماتریس درهم‌ریختگی مدل FEAT برای بخش آموزش

این نتایج تأیید می‌کنند که استفاده از Transformer در FEAT، مزیت خود را در تحلیل عمیق‌تر ساختار داده‌ها و روابط بین کلاس‌ها نشان می‌دهد.

۴-۵- تحلیل عملکرد مدل‌ها در مرحله Test و Validation

پس از اتمام فرآیند آموزش اپیزودمحور، عملکرد مدل‌ها در مرحله اعتبارسنجی و آزمون مورد بررسی قرار می‌گیرد. از آن‌جا که داده‌های این مرحله در فرآیند یادگیری استفاده نشده‌اند، این تحلیل بیانگر توانایی تعمیم مدل‌ها به اپیزودهای نادیده است.

۴-۵-۱- تحلیل Validation Loss در سطح اپیزود

در مرحله اعتبارسنجی، که معیار اصلی ارزیابی توان تعمیم مدل‌ها محسوب می‌شود، هر دو مدل عملکرد قابل توجهی از خود نشان می‌دهند. نمودار Validation Loss بیانگر آن است که مقادیر زیان در هر دو مدل در سطح پایین و نسبتاً پایدار باقی می‌ماند، که نشان‌دهنده‌ی عدم بروز بیش‌برازش شدید است. با این وجود، مدل ProtoNet نوسانات کمتری در مقدار زیان اعتبارسنجی دارد و رفتار هموارتری از خود نشان می‌دهد.

نکته‌ی مهم در تحلیل مشترک دو نمودار آن است که افزایش Validation Loss در FEAT لزوماً با کاهش شدید Accuracy همراه نیست. این موضوع نشان می‌دهد که FEAT در برخی موارد با اعتماد کمتر (confidence پایین‌تر) پیش‌بینی صحیح انجام می‌دهد. در مقابل ProtoNet با وجود Loss کمتر و پایدارتر، به سطح دقت مشابهی با FEAT دست می‌یابد. این امر نشان‌دهنده‌ی تصمیم‌گیری ساده‌تر اما قابل اعتمادتر در سطح احتمال است.

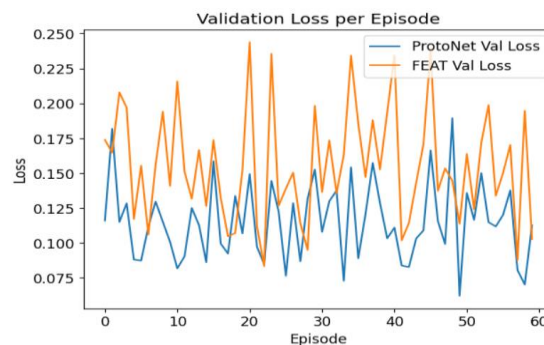
۴-۶- تحلیل جامع گزارش‌های طبقه‌بندی ProtoNet و FEAT

گزارش‌های طبقه‌بندی ارائه‌شده تصویری دقیق از رفتار دو مدل ProtoNet و FEAT در سطح کلاس‌های منفرد و همچنین در مقیاس کلی مجموعه آزمون فراهم می‌کنند. این گزارش‌ها با ارائه معیارهای precision، recall و F1-score برای هر کلاس، امکان تحلیل متوازن‌تری از عملکرد مدل‌ها فراتر از معیار کلی دقت را فراهم می‌سازند، که این موضوع به‌ویژه در مسائل Few-Shot Learning از اهمیت بالایی برخوردار است.

۴-۶-۱- معیارهای ارزیابی برای conv4

در مدل ProtoNet، مقادیر precision و recall در تمامی کلاس‌ها در بازه‌ای بسیار نزدیک به یکدیگر و حول مقدار ۰.۹۵ متمرکز شده‌اند. این همگرایی بالا میان precision و recall نشان‌دهنده‌ی آن است که مدل نه تنها در شناسایی صحیح نمونه‌های هر کلاس موفق عمل می‌کند، بلکه از تولید پیش‌بینی‌های اشتباه به‌صورت متوازن اجتناب می‌نماید. به بیان دیگر، ProtoNet تعادل مناسبی میان خطاهای نوع اول (False Positive) و نوع دوم (False Negative) برقرار کرده است. این رفتار با ماهیت پروتوتایپی مدل سازگار است، چرا که تصمیم‌گیری بر اساس فاصله تا مرکز هر کلاس موجب شکل‌گیری مرزهای تصمیم نسبتاً پایدار و کم‌نوسان می‌شود.

تحلیل جزئی‌تر کلاس‌ها نشان می‌دهد که نوسانات بسیار اندکی میان کلاس‌ها وجود دارد. به‌عنوان مثال، کلاس‌هایی با recall اندکی بالاتر (نظیر کلاس‌های ۲ و ۶) نشان می‌دهند که نمونه‌های این کلاس‌ها در فضای ویژگی خوشه‌بندی فشرده‌تری داشته‌اند، در حالی که در کلاس‌هایی با precision یا recall کمی پایین‌تر (مانند کلاس ۴)، احتمال همپوشانی جزئی ویژگی‌ها با کلاس‌های مجاور وجود دارد. با این حال، این اختلاف‌ها بسیار محدود بوده و تأثیر معناداری بر عملکرد کلی مدل ندارند.

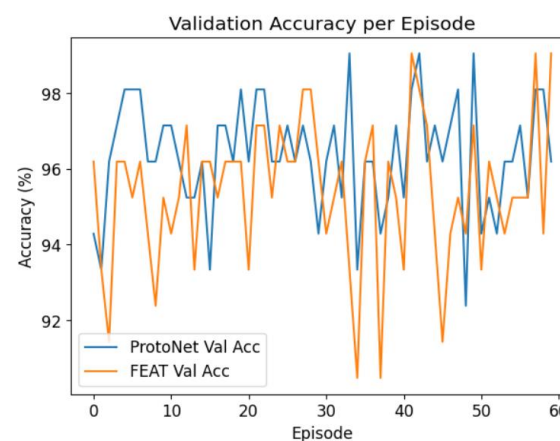


عکس ۴-۵- ۱ نمودار تحلیل رفتار مدل برای متغیر ارزیابی Loss بر روی داده‌های validation

این پایداری را می‌توان به ساختار ساده‌تر تابع تصمیم و استفاده از فاصله اقلیدسی نسبت داد که موجب کاهش حساسیت مدل به تغییرات اپیزودمحور می‌شود. در مقابل، مدل FEAT با وجود دستیابی به مقادیر زبان مشابه، نوسانات بیشتری را تجربه می‌کند که ناشی از تطبیق پویا پروتوتایپ‌ها و وابستگی شدیدتر به ترکیب نمونه‌های هر اپیزود است.

۴-۵-۲- تحلیل Validation Accuracy در سطح اپیزود

تحلیل نمودار Validation Accuracy نشان می‌دهد که هر دو مدل در بازه‌ی دقت بالا قرار دارند و توانایی تعمیم مناسبی به اپیزودهای نادیده از خود نشان می‌دهند. مدل ProtoNet دقتی نسبتاً یکنواخت‌تر در طول اپیزودها ارائه می‌دهد، در حالی که مدل FEAT در برخی اپیزودها به دقت بالاتری دست می‌یابد و در برخی موارد افت‌های مقطعی مشاهده می‌شود. این رفتار نشان می‌دهد که FEAT در اپیزودهایی که ساختار بین‌کلاسی غنی‌تری دارند، قادر به بهره‌برداری مؤثرتری از روابط نمونه‌هاست، اما در اپیزودهای چالش‌برانگیزتر، حساسیت بیشتری از خود نشان می‌دهد.



عکس ۴-۵- ۲ نمودار تحلیل رفتار مدل برای متغیر ارزیابی Accuracy بر روی داده‌های validation

در نتیجه، هر دو مدل از منظر کیفیت کلی پیش‌بینی در سطح کلاس‌های منفرد، عملکردی پایدار و قابل اعتماد ارائه می‌دهند.

۵- نتیجه‌گیری

یافته‌های این پژوهش تأیید می‌کند که استفاده از یادگیری کم‌نمونه در حوزه‌ی تشخیص رفتار راننده، نه تنها یک انتخاب جایگزین در نبود داده‌های کافی، بلکه یک رویکرد هدفمند و کارآمد برای طراحی سامانه‌های ایمنی هوشمند است. نتایج نشان می‌دهد که ترکیب استخراج‌گرهای سبک کانولوشنی با روش‌های متا-یادگیری و سازوکارهای توجه می‌تواند تعادلی مناسب میان دقت، پایداری و قابلیت تعمیم ایجاد کند؛ تعادلی که برای کاربردهای بلادرنگ و ایمنی‌محور اهمیت حیاتی دارد.

در نهایت، این مطالعه نشان می‌دهد که مدل‌های مبتنی بر Few-Shot Learning، به‌ویژه روش‌های تطبیقی مانند FEAT، ظرفیت بالایی برای توسعه‌ی سامانه‌های تشخیص رفتار راننده با حداقل نیاز به داده‌ی برچسب‌خورده دارند. این نتایج می‌تواند مسیر را برای پژوهش‌های آتی در جهت استفاده از معماری‌های عمیق‌تر، ترکیب داده‌های چندوجهی و بررسی سناریوهای پیچیده‌تر رفتاری هموار سازد و گامی مؤثر در جهت طراحی سامانه‌های حمل‌ونقل هوشمند، ایمن و مقیاس‌پذیر باشد.

مراجع

- [1] J.Snell, K.Swersky, R.Zemel, "Prototypical Networks for Few-shot Learning"
- [2] H.Ye, H.Hu, D.Zhan, F.Sha, "Few-Shot Learning via Embedding Adaptation with Set-to-Set Function"
- [3] D.Ramanan, F.Moura, Y.Gui, "Few-shot human motion prediction via meta-learning"
- [4] K.Simonyan, A.Zisserman, "Deep convolutional networks for large-scale image recognition"
- [5] M.Shahverdi, M.Fathi, "Estimating accident risk based on driver behavior using machine learning algorithms"
- [6] A.Vaswani, N.Shazeer, N.Parmar, J.Uskoreit, L.Jones, L.Kaiser, "Attention is all you need"

==== ProtoNet Classification Report ====				
	precision	recall	f1-score	support
0	0.95	0.95	0.95	1200
1	0.95	0.95	0.95	1200
2	0.94	0.96	0.95	1200
3	0.96	0.95	0.96	1200
4	0.96	0.94	0.95	1200
5	0.95	0.95	0.95	1200
6	0.96	0.96	0.96	1200
accuracy			0.95	8400
macro avg	0.95	0.95	0.95	8400
weighted avg	0.95	0.95	0.95	8400

عکس ۴-۶-۱- جدول معیارهای ارزیابی برای مدل Conv4

۴-۶-۲- معیارهای ارزیابی برای Feat

در مدل FEAT، الگوی کلی عملکرد بسیار نزدیک به ProtoNet است، اما تفاوت‌های ظریفی در توزیع خطاها مشاهده می‌شود. مقادیر precision و recall در FEAT نیز در محدوده‌ی ۰.۹۴ تا ۰.۹۶ قرار دارند که بیانگر عملکرد رقابتی و پایدار این مدل است. با این وجود، در برخی کلاس‌ها مشاهده می‌شود که precision اندکی کمتر و recall اندکی بیشتر از ProtoNet است یا بالعکس. این رفتار نشان‌دهنده‌ی ماهیت تطبیقی FEAT است که با تنظیم پویا پروتوتایپ‌ها بر اساس روابط درون‌پایزودی، در برخی کلاس‌ها تمایل بیشتری به جذب نمونه‌های مرزی دارد.

==== FEAT Classification Report ====				
	precision	recall	f1-score	support
0	0.95	0.94	0.95	1200
1	0.95	0.95	0.95	1200
2	0.95	0.95	0.95	1200
3	0.96	0.95	0.96	1200
4	0.94	0.95	0.95	1200
...				
accuracy			0.95	8400
macro avg	0.95	0.95	0.95	8400
weighted avg	0.95	0.95	0.95	8400

عکس ۴-۶-۲- جدول معیارهای ارزیابی برای مدل Feat

نکته‌ی حائز اهمیت آن است که مقدار F1-score در هر دو مدل برای تمامی کلاس‌ها تقریباً ثابت و برابر با ۰.۹۵ یا ۰.۹۶ است. این موضوع نشان می‌دهد که حتی در مواردی که precision و recall به‌صورت نامتقارن تغییر کرده‌اند، تعادل کلی میان این دو معیار حفظ شده است.